Smart MCQ Solver: A Comparative Study of Retrieval, Ranking, and Transformer-Based Approaches

BSDA2001P — Introduction to DL and GenAI Project Report

Suchintika Sarkar -21F3002441

Kaggle Competition: smart-mcq-solver-challenge

1. Abstract / Executive Summary

This project tackles the smart-mcq-solver-challenge Kaggle competition, a five-option multiple-choice-question (MCQ) answering task over 2,000 training and 500 test questions drawn from mixed academic domains (physics, philosophy, general science). The objective was to design, train, and compare complementary modelling strategies spanning classical feature-engineered ranking, fine-tuned transformer classification, and retrieval-augmented ranking, and to combine them into a final ensemble. Four systems were built: (1) a TF-IDF + LightGBM learning-to-rank model, (2) a fine-tuned microsoft/deberta-v3-base transformer using the Hugging Face AutoModelForMultipleChoice head, (3) a retrieval-augmented ranking pipeline (RAG) pairing a MiniLM bi-encoder retriever and a MS-MARCO cross-encoder reranker with a LightGBM ranker over TF-IDF and contextual features, and (4) a weighted ensemble of the three. All experiments were tracked with Weights & Biases. Cross-validation/held-out MAP@3 was 0.9910 (Model 1), 0.9438 (Model 2, after fixing the random seed and extending training), and 0.9975 (Model 3), while the Kaggle public leaderboard showed a consistently lower score for all three (0.7550, 0.7260, 0.7610) — a gap of comparable size (~0.22–0.24) across every model, indicating a general overfitting/distribution-shift effect rather than an issue specific to one modelling approach. The RAG-based ranker was the strongest single model on the leaderboard, and the weighted ensemble (0.35 / 0.20 / 0.45) was used for the final submission.

2. Introduction

2.1 Problem Statement

The competition provides a train.csv of 2,000 and a test.csv of 500 MCQ items, each with a prompt (question stem), five answer options (A–E), and, for the training set, the correct option label. The task is to predict, for every test question, a ranked list of the top-3 most likely correct options. Submissions are scored with Mean Average Precision at 3 (MAP@3), which rewards placing the correct answer as high as possible within the top three guesses.

2.2 Project Objective

The goals of this project were to (a) build at least three materially different modelling approaches to the same ranking problem — a classical ML pipeline, a fine-tuned transformer, and a retrieval-augmented pipeline — (b) gain hands-on experience with the Hugging Face ecosystem (tokenizers, AutoModelForMultipleChoice, Trainer), sentence-transformer retrieval, and gradient-boosted learning-to-rank, (c) track and compare all experiments systematically using Weights & Biases, and (d) combine the individual models into an ensemble to maximise leaderboard performance.

2.3 Report Structure

Section 3 describes the dataset and preprocessing pipeline. Section 4 covers the tokenization strategies used across models. Section 5 details the architecture and training strategy of each of the four models built. Section 6 compares their performance using cross-validation and leaderboard metrics. Section 7 concludes with key learnings, challenges, and future work, followed by references in Section 8.

3. Dataset & Preprocessing

3.1 Dataset Description

The training set contains 2,000 rows and 8 columns (id, prompt, A, B, C, D, E, answer); the test set contains 500 rows with the same columns minus the answer label. There are no missing values in either split. Each question has exactly one correct option among five, so a naive random baseline would score roughly 0.28 MAP@3 (weighting partial credit for 2nd/3rd-place guesses).

3.2 Exploratory Data Analysis

The correct-answer label is close to uniformly distributed across A–E (ruling out a majority-class shortcut); prompt length is right-skewed (most prompts 40–100 characters); and prompts are consistently longer and more variable than their options — which informed the max-sequence-length choice in Section 4:

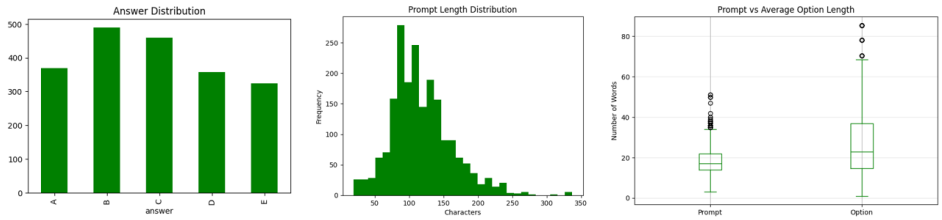


Figure 1. Answer distribution, prompt-length distribution, and prompt-vs-option word count (left to right).

A word-frequency analysis of the prompts (after lowercasing, punctuation removal, and English stop-word filtering) showed the corpus spans multiple academic subject areas — physics/cosmology terms ("fusion", "relativity", "light") and philosophy/epistemology terms ("Heidegger", "existence", "time") were both prominent, confirming the dataset is domain-diverse rather than single-topic.

3.3 Data Preprocessing

- Wide-to-long reshaping: every question was expanded from one wide row (prompt + 5 options) into 5 long rows (one per option), each carrying a binary label indicating whether that option is correct. This converts the MCQ problem into a pointwise/pairwise ranking problem suitable for LightGBM's learning-to-rank objective.
- Prompt cleaning for retrieval: templated question-stem phrases (e.g., "Pick the best possible answer:", "Determine the correct option:") were stripped before building the retrieval knowledge base, so that semantic retrieval was not biased by boilerplate instruction phrasing.
- Feature engineering (classical models): prompt/option character and word counts, prompt–option word-overlap count and ratio, and TF-IDF cosine similarity between prompt and option were computed as auxiliary numerical features.
- Train/validation split: GroupKFold (5 folds, grouped by question id) was used for the TF-IDF/LightGBM and RAG models to avoid leaking options from the same question across folds; a stratified 80/20 train/validation split (stratified on the answer label) was used for the DeBERTa fine-tuning run.

3.4 Data Augmentation

No explicit data augmentation (paraphrasing, back-translation, etc.) was applied. The main form of "augmentation" used was retrieval-based context injection for Model 3, described in Section 5.3.

4. Tokenization Strategy

4.1 Primary Tokenizer Selection

The tokenizer used for the best single-model system, the retrieval-augmented LightGBM ranker, is a hybrid of word-level and character-level TF-IDF vectorization; the tokenizer used for the transformer model (Model 2) is the SentencePiece-based tokenizer shipped with microsoft/deberta-v3-base, which uses a 128k-vocabulary unigram/BPE-style subword model shared across DeBERTa-v3's disentangled-attention encoder.

4.2 Justification

DeBERTa-v3's SentencePiece tokenizer operates directly on raw text (no whitespace pre-tokenization required) and handles the mixed technical vocabulary (physics/philosophy terminology, proper nouns such as "Heidegger") gracefully by falling back to subword units, avoiding out-of-vocabulary issues that a fixed word-level vocabulary would face. For the classical models, TF-IDF word n-grams (1–2) capture topical/lexical overlap between prompt and option, while character n-grams (3–6) are robust to minor spelling/morphological variation and reward options that literally reuse phrasing from the prompt — a useful signal given how MCQ distractors are typically constructed.

4.3 Implementation Details

- Transformer pipeline: Hugging Face AutoTokenizer paired each prompt with each of the 5 options (first_sentence = prompt × 5, second_sentence = [A, B, C, D, E]), with truncation=True and max_length=256, then a custom DataCollatorForMultipleChoice dynamically padded and reshaped the batch to (batch_size, 5, seq_len) for AutoModelForMultipleChoice.
- Classical/RAG pipeline: scikit-learn's TfidfVectorizer was fit separately for word n-grams (max_features=50,000, min_df=2, max_df=0.95, sublinear_tf=True, English stop-words removed) and character n-grams (analyzer="char_wb", ngram_range=(3,6), max_features=30,000), then horizontally stacked with numeric features via scipy's hstack into a single sparse design matrix for LightGBM.
- Retrieval pipeline: questions and knowledge-base documents were encoded with the sentence-transformers/all-MiniLM-L6-v2 bi-encoder (384-dim, L2-normalized embeddings) for fast FAISS approximate search, and the top candidates were rescored with the cross-encoder/ms-marco-MiniLM-L-6-v2 cross-encoder for higher-precision reranking.

4.4 Experimental Comparison

The appendix milestones compare a pure TF-IDF cosine-similarity pipeline against an all-MiniLM-L6-v2 dense-embedding pipeline for zero-shot option ranking. The dense-embedding pipeline recovered a meaningful number of questions that the sparse TF-IDF pipeline missed, confirming that semantic (embedding-based) similarity captures paraphrase-level matches that lexical overlap

cannot, which motivated combining both signal types (TF-IDF and dense retrieval) in the final RAG + LightGBM system rather than relying on either alone.

5. Modeling & Experimentation

Four systems were trained and logged as separate Weights & Biases runs. All models are evaluated on the same MAP@3-oriented protocol described in Section 6.

5.1 Model 1: TF-IDF + LightGBM Learning-to-Rank

Architecture. Model 1 reframes MCQ answering as a pointwise/learning-to-rank problem: each (prompt, option) pair becomes one row with a sparse feature vector — 50,000-dim word TF-IDF, 30,000-dim character TF-IDF, and 7 hand-engineered numeric features (lengths, word overlap, cosine similarity) — 41,251 features per row. Two variants were trained: an LGBMClassifier baseline (binary relevance objective, evaluated with accuracy/F1/AUC as well as MAP@3), and an LGBMRanker (lambdarank objective, NDCG metric) as the improved variant, since ranking objectives more directly optimise MAP@3 than a pointwise classifier.

Training strategy & salient points. 5-fold GroupKFold cross-validation (grouped by question id, 500 estimators, learning rate 0.05, 63 leaves) prevented options from the same question leaking across folds. Requiring no GPU and training in seconds per fold, this pipeline is a fast, interpretable baseline whose feature importances directly show which lexical/overlap signals drive the ranking.

5.2 Model 2: Fine-Tuned DeBERTa-v3-Base (Transformer)

Architecture. microsoft/deberta-v3-base (He et al., 2021) is a 12-layer, 768-hidden, 12-head transformer encoder pretrained with disentangled attention (content and relative-position representations encoded and attended to separately) and an ELECTRA-style replaced-token-detection objective, giving stronger sample efficiency than BERT/RoBERTa. It was loaded via Hugging Face's AutoModelForMultipleChoice, which adds a linear head over the pooled [CLS] representation of each of the 5 (prompt, option) pairs and applies softmax across choices. The disentangled attention is well suited to this task, where the correct option is often distinguished by subtle rewording rather than distinct vocabulary.

Fine-tuning strategy. All parameters were fine-tuned (no frozen layers) for 6 epochs, AdamW, learning rate 2e-5, weight decay 0.01, per-device batch size 4 with gradient accumulation of 2 (effective batch size 8), on an 80/20 stratified split (1,600/400 questions), Tesla T4 GPU. An explicit seed (42) was set before model init (cuDNN determinism enabled, seed=42/data_seed=42 in TrainingArguments) — an earlier unseeded 4-epoch run had validation MAP@3 vary between 0.61 and 0.89 across reruns purely from random head-init/data order; seeding plus 2 extra epochs stabilised this at 0.94. Best checkpoint selected by lowest eval_loss. No quantization — full float32.

5.3 Model 3: Retrieval-Augmented Ranking (RAG)

Two iterations were built. The baseline RAG pipeline retrieved the top-5 nearest knowledge-base passages per question with a MiniLM bi-encoder + FAISS flat inner-product index, concatenated them into a single context string, and directly reranked the 5 answer options against that context using the MS-MARCO cross-encoder — with no learned ranking model on top. This worked but had no mechanism to combine retrieval signal with lexical/statistical features.

The improved pipeline addressed this by retrieving a larger candidate pool (top-20), reranking with the cross-encoder to select a single best-supporting passage per question, and then feeding that passage — concatenated with the prompt and option — into the same TF-IDF word/char featurization used in Model 1, plus two RAG-specific features (context–option word overlap and option length), before training an LGBMRanker (lambdarank, MAP metric, 500 estimators) on the combined feature set.

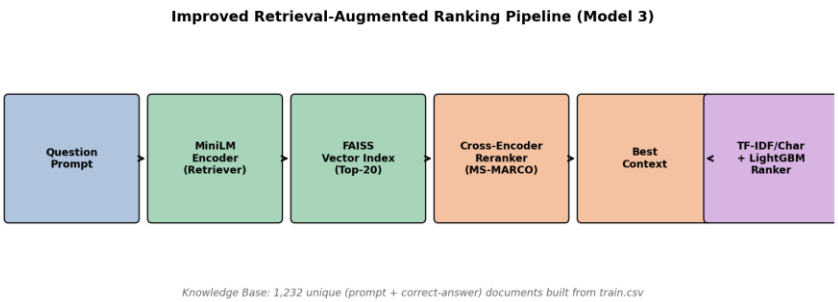


Figure 4. Improved retrieval-augmented ranking pipeline (Model 3).

Salient points. The knowledge base itself was built by concatenating each training question's cleaned prompt with its correct-answer text (1,232 unique documents after de-duplication), so retrieval is effectively looking up semantically similar questions the model has already "seen" the answer to — a design choice that boosts cross-validation scores but, as discussed in Section 6, does not transfer as strongly to genuinely unseen test questions.

5.4 Model 4: Weighted Ensemble

The final model is a weighted average of the softmax-normalized option scores from Models 1, 2, and 3: 0.35 × Model 1 + 0.20 × Model 2 + 0.45 × Model 3, with weights set roughly in proportion to each model's individual Kaggle public-leaderboard score (Model 3 > Model 1 > Model 2). The top-3 options by ensembled probability form the final submission.

6. Performance & Comparative Analysis

6.1 Evaluation Metrics

The competition's primary metric is MAP@3. For Model 1's classifier variant, accuracy, F1-score, and ROC-AUC were also tracked per fold as secondary diagnostics of the underlying binary relevance model.

6.2 Training / Validation Performance

5-fold GroupKFold MAP@3 for the LightGBM ranker (Model 1, improved variant):

Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Mean
0.9850	0.9925	0.9938	0.9892	0.9946	0.9910

5-fold GroupKFold MAP@3 for the improved RAG + LightGBM ranker (Model 3):

Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Mean
0.9963	0.9988	0.9963	0.9975	0.9988	0.9975

DeBERTa-v3 (Model 2) reached a held-out validation MAP@3 of 0.9438 after fixing the training seed and extending to 6 epochs (up from an unstable 0.61–0.89 range across unseeded reruns at 4 epochs; see Section 5.2) — though as Section 6.3 shows, this did not translate into a higher leaderboard score.

6.3 Comparative Report

All logged runs are tracked in the Weights & Biases project 21f3002441-t22026. The table and chart below compare cross-validation/held-out performance against actual Kaggle public-leaderboard MAP@3:

Model	Approach	CV / Val MAP@3	Public LB MAP@3	Relative Cost
Model 1	TF-IDF + LightGBM Ranker	0.9910	0.7550	Low (CPU, seconds/fold)
Model 2	Fine-tuned DeBERTa-v3	0.9438	0.7260	High (GPU, ~9 min)
Model 3	RAG (MiniLM + Cross-Encoder + LightGBM)	0.9975	0.7610	Medium-High (GPU embed + rerank)
Model 4	Weighted Ensemble (0.35 / 0.20 / 0.45)	—	Final submission	Low (inference-only)

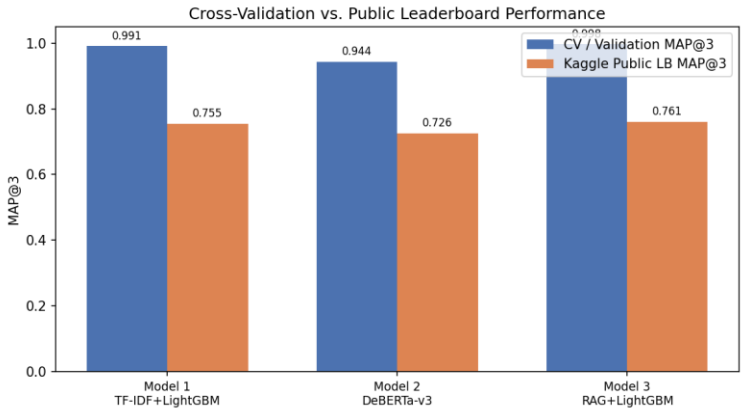


Figure 5. Cross-validation vs. Kaggle public-leaderboard MAP@3 by model.

The striking result is that all three models show a gap of similar size (~0.22–0.24 MAP@3) between their cross-validation/validation score and the Kaggle public leaderboard. This gap was initially attributed to the TF-IDF vocabulary and RAG knowledge base being built directly from training prompts/answers (Models 1 and 3 only). However, after fixing DeBERTa's random seed and extending training, its validation MAP@3 rose from 0.6067 to 0.9438 while its leaderboard score stayed essentially flat (0.7303 → 0.7260). Since DeBERTa shares no TF-IDF vocabulary or retrieval knowledge base with Models 1 and 3, this rules out lexical/retrieval leakage as the sole explanation. The more likely combined cause is (a) a genuine distribution difference between the training split and Kaggle's held-out test questions, and (b) all three models overfitting to a relatively small validation signal once trained hard enough

to fit it closely. Cross-validation/validation performance on this dataset should therefore be treated as an optimistic upper bound, not a reliable estimate of leaderboard performance, regardless of model family.

6.4 Kaggle Performance

Public leaderboard MAP@3 scores logged for the individual models were 0.7550 (Model 1), 0.7260 (Model 2), and 0.7610 (Model 3, the strongest single model). The weighted ensemble (Model 4) was submitted as the final entry; exact leaderboard rank was not captured in the notebook and should be filled in from the competition's leaderboard page for the final submission.

7. Conclusion & Future Work

7.1 Key Learnings

- A high cross-validation/validation score is not proof of generalisation. All three models — including DeBERTa, which shares no vocabulary or knowledge base with the others — showed a similar-sized gap (~0.22–0.24 MAP@3) to the leaderboard, pointing to general small-sample overfitting rather than an issue specific to one model family.
- Run-to-run instability in transformer fine-tuning is a measurable risk on small data: DeBERTa's validation MAP@3 ranged 0.61–0.89 across unseeded reruns before seeding + 2 extra epochs stabilised it at 0.94 — but that stability alone did not improve the leaderboard score, so reproducibility and generalisation are separate problems.
- Retrieval-augmented ranking remained the strongest single approach on the leaderboard even after accounting for the generalisation gap seen across all models.

7.2 Challenges Faced

- GPU memory on the Kaggle T4 instance limited the DeBERTa per-device batch size to 4, requiring gradient accumulation.
- DeBERTa fine-tuning was highly sensitive to random initialization/data order until seeding was added explicitly, making early runs hard to compare.
- The retrieval knowledge base includes near-duplicates of the query itself, which likely adds to the general gap for Models 1 and 3.

7.3 Areas for Improvement

- Use k-fold cross-validation for DeBERTa instead of one 80/20 split, and report mean ± std across seeds.
- Exclude each question's own document from Model 3's retrieval results to isolate leakage from distribution shift.
- Add a genuinely different architecture family (custom BiLSTM/text-CNN or RNN) as a diverse ensemble component.
- Tune ensemble weights via a held-out stacking model or Optuna search instead of setting them manually.

8. References

He, P., Liu, X., Gao, J., & Chen, W. (2021). DeBERTa: Decoding-enhanced BERT with Disentangled Attention. ICLR 2021.

He, P., Gao, J., & Chen, W. (2021). DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing. arXiv:2111.09543.

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. NeurIPS 2017.

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP 2019.

Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. IEEE Transactions on Big Data (FAISS library).

Nogueira, R., & Cho, K. (2019). Passage Re-ranking with BERT. arXiv:1901.04085 (basis for the ms-marco-MiniLM cross-encoder family).

Hugging Face Transformers, Datasets, and Evaluate library documentation — https://huggingface.co/docs

Weights & Biases experiment tracking documentation — https://docs.wandb.ai

microsoft/deberta-v3-base model card — https://huggingface.co/microsoft/deberta-v3-base

sentence-transformers/all-MiniLM-L6-v2 and cross-encoder/ms-marco-MiniLM-L-6-v2 model cards — https://huggingface.co/sentence-transformers, https://huggingface.co/cross-encoder

Kaggle competition: smart-mcq-solver-challenge (course-hosted competition, BSDA2001P).